

INFORME TÉCNICO

Backend REST API – Gestión de Tareas (Todo App)

1. Descripción General

El proyecto es una API REST para gestión de tareas personales (Todo App), desarrollada en Node.js con el framework Express 5. Implementa una arquitectura MVC (Modelo-Vista-Controlador) con conexión a base de datos MySQL a través de un pool de conexiones.

La aplicación permite administrar usuarios y sus tareas, incluyendo filtrado por estado y prioridad, además de un módulo de estadísticas con métricas tanto globales como por usuario.

2. Stack Tecnológico y Dependencias

2.1 Dependencias de Producción

Paquete	Versión	Función
express	^5.2.1	Framework HTTP principal
mysql2	^3.16.3	Driver MySQL con soporte Promises y pool
cors	^2.8.6	Habilita Cross-Origin Resource Sharing
dotenv	^17.2.4	Carga variables de entorno desde .env
express-validator	^7.3.1	Validación de datos de entrada (instalado, no usado aún)

2.2 Dependencias de Desarrollo

Paquete	Versión	Función
nodemon	^3.1.11	Recarga automática del servidor en desarrollo

2.3 Scripts npm

Comando	Descripción
npm run dev	Inicia con nodemon (hot-reload para desarrollo)
npm start	Inicia con node directamente (producción)

3. Estructura del Proyecto

El proyecto sigue una estructura MVC organizada de la siguiente manera:

```
backend/  
├─ index.js                # Punto de entrada principal  
├─ package.json            # Metadatos y dependencias  
├─ .env / .env.example     # Variables de entorno  
├─ database.sql            # Script DDL de la base de datos  
└─ src/  
    ├─ config/  
    │   └─ db.js           # Pool de conexiones MySQL  
    ├─ controllers/  
    │   ├─ tareas.controller.js  
    │   ├─ usuarios.controller.js  
    │   └─ estadisticas.controller.js  
    └─ routes/  
        ├─ tareas.routes.js  
        ├─ usuarios.routes.js  
        └─ estadisticas.routes.js
```

4. Modelo de Base de Datos

La base de datos se denomina todo_app y contiene dos tablas relacionadas:

```
backend > src > config > JS db.js > ...  
1  const mysql = require("mysql2/promise");  
2  require("dotenv").config();  
3  
4  const pool = mysql.createPool({  
5    host: process.env.DB_HOST,  
6    user: process.env.DB_USER,  
7    password: process.env.DB_PASSWORD,  
8    database: process.env.DB_NAME,  
9    port: process.env.DB_PORT,  
10   waitForConnections: true,  
11   connectionLimit: 10,  
12   queueLimit: 0  
13 });  
14  
15 module.exports = pool;  
16
```

4.1 Tabla: usuarios

Campo	Tipo	Restricciones
id	INT AUTO_INCREMENT	PRIMARY KEY
nombre	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE NOT NULL
fecha_creacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

4.2 Tabla: tareas

Campo	Tipo	Restricciones / Notas
id	INT AUTO_INCREMENT	PRIMARY KEY
titulo	VARCHAR(200)	NOT NULL
descripcion	TEXT	Opcional
estado	ENUM	'pendiente' 'en_progreso' 'completada'. Default: pendiente
prioridad	ENUM	'baja' 'media' 'alta'. Default: media
fecha_vencimiento	DATE	Opcional
usuario_id	INT	FK → usuarios(id) ON DELETE CASCADE
fecha_creacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
fecha_actualizacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

```
backend > src > controllers > JS tareas.controller.js > crearTarea > error
1  const pool = require("../config/db");
2
3  const esFechaPasada = (fechaStr) => {
4    if (!fechaStr) return false;
5
6    const hoy = new Date();
7    hoy.setHours(0, 0, 0, 0);
8
9    const fecha = new Date(fechaStr);
10   fecha.setHours(0, 0, 0, 0);
11
12   return fecha < hoy;
13 };
14
15 const usuarioExiste = async (usuarioId) => {
16   const [rows] = await pool.query("SELECT id FROM usuarios WHERE id = ?", [usuarioId]);
17   return rows.length > 0;
18 };
19
20 const crearTarea = async (req, res) => {
21   try {
22     const { titulo, descripcion, estado, prioridad, fecha_vencimiento, usuario_id } = req.body;
23
24     if (!titulo || titulo.trim() === "") {
25       return res.status(400).json({ error: "El título es obligatorio" });
26     }
27
28     if (!usuario_id) {
29       return res.status(400).json({ error: "usuario_id es obligatorio" });
30     }
31
32     if (!(await usuarioExiste(usuario_id))) {
33       return res.status(400).json({ error: "El usuario_id no existe" });
34     }
35
36     if (fecha_vencimiento && esFechaPasada(fecha_vencimiento)) {
37       return res.status(400).json({ error: "La fecha_vencimiento no puede ser pasada" });
38     }
39
40     // ... resto del código para crear la tarea ...
41   } catch (error) {
42     // ... manejo de errores ...
43   }
44 }
```

Lín. 58, Col. 38 (1 seleccionada) Espacios: 2 UTF-8 LF JavaScript Iniciar sesión Go Live

La relación entre tablas es 1:N (un usuario puede tener muchas tareas). La clave foránea implementa eliminación en cascada (ON DELETE CASCADE), de modo que al eliminar un usuario se eliminan automáticamente todas sus tareas.

5. Configuración y Conexión a BD

La conexión a MySQL se maneja mediante un pool de conexiones creado en `src/config/db.js` usando `mysql2/promise`. Esta elección permite usar `async/await` de forma nativa en los controladores, evitando el uso de `callbacks`.

Las credenciales son cargadas desde variables de entorno vía `dotenv`, siguiendo buenas prácticas de seguridad. El archivo `.env.example` define las variables requeridas:

```
backend > ⚙ .env.example
1  PORT=3001
2  DB_HOST=localhost
3  DB_USER=root
4  DB_PASSWORD=*****
5  DB_NAME=todo_app
6  DB_PORT=3306
7
```

El pool está configurado con `connectionLimit: 10`, lo que permite hasta 10 conexiones simultáneas, adecuado para un entorno de desarrollo o carga moderada.

6. Endpoints de la API

6.1 Usuarios – /api/usuarios

Método	Ruta	Función	Código Exitoso
POST	/api/usuarios	Crear nuevo usuario	201 Created
GET	/api/usuarios	Listar todos los usuarios	200 OK

6.2 Tareas – /api/tareas

Método	Ruta	Función	Código Exitoso
POST	/api/tareas	Crear nueva tarea	201 Created
GET	/api/tareas	Listar tareas (con filtros opcionales)	200 OK
GET	/api/tareas/:id	Obtener tarea por ID	200 OK
PUT	/api/tareas/:id	Actualizar tarea completa	200 OK
DELETE	/api/tareas/:id	Eliminar tarea	200 OK

6.3 Estadísticas – /api/estadisticas

Método	Ruta	Función	Código Exitoso
GET	/api/estadisticas	Estadísticas globales de todas las tareas	200 OK
GET	/api/estadisticas/:usuario_id	Estadísticas de un usuario específico	200 OK

6.4 Utilidades

Método	Ruta	Función
GET	/	Health check básico – responde con texto
GET	/api/health	Verifica conexión con la base de datos MySQL

7. Lógica de Controladores

7.1 Controlador de Usuarios

Implementa dos funciones principales:

- crearUsuario: valida que nombre y email no estén vacíos, inserta el registro y captura el error ER_DUP_ENTRY de MySQL para responder con un mensaje amigable cuando el email ya existe.
- obtenerUsuarios: devuelve todos los usuarios sin filtrado. No incluye paginación.

7.2 Controlador de Tareas

Es el módulo más completo. Contiene validaciones de negocio relevantes:

- Validación de campos obligatorios (titulo, usuario_id) antes de insertar.
- Verificación de existencia del usuario_id en la BD antes de crear o actualizar una tarea (función auxiliar usuarioExiste).
- Validación de fechas: la función auxiliar esFechaPasada compara la fecha_vencimiento con la fecha actual, rechazando fechas pasadas.
- Soporte de filtros en el listado: acepta query params estado, prioridad y usuario_id para construir dinámicamente la cláusula WHERE.
- Actualización parcial inteligente: si un campo no viene en el cuerpo, conserva el valor actual de la BD (patrón merge manual).

7.3 Controlador de Estadísticas

Genera métricas calculadas directamente en SQL mediante agregaciones GROUP BY.

Devuelve:

Campo	Descripción
por_estado	Cantidad de tareas agrupadas por estado (pendiente, en_progreso, completada)
por_prioridad	Cantidad de tareas agrupadas por prioridad (baja, media, alta)
vencidas	Tareas con fecha_vencimiento pasada y estado distinto a 'completada'
promedio_dias_completar	Promedio de días entre creación y última actualización de tareas completadas

Nota técnica: El campo promedio_dias_completar es una aproximación, ya que el esquema no posee un campo fecha_completada dedicado; se usa fecha_actualizacion como sustituto.

8. Flujo de una Petición HTTP

El flujo estándar de cualquier petición a la API sigue estos pasos:

Paso	Componente	Descripción
1	Cliente HTTP	Envía petición (Postman, navegador, frontend)
2	index.js	Middlewares globales: CORS, JSON parser, enrutamiento
3	*.routes.js	Identifica el handler según método + ruta
4	*.controller.js	Ejecuta lógica de negocio y validaciones
5	src/config/db.js	Pool de conexiones obtiene una conexión MySQL
6	MySQL	Ejecuta la query y devuelve resultados
7	*.controller.js	Formatea la respuesta JSON
8	Cliente HTTP	Recibe respuesta con status code apropiado